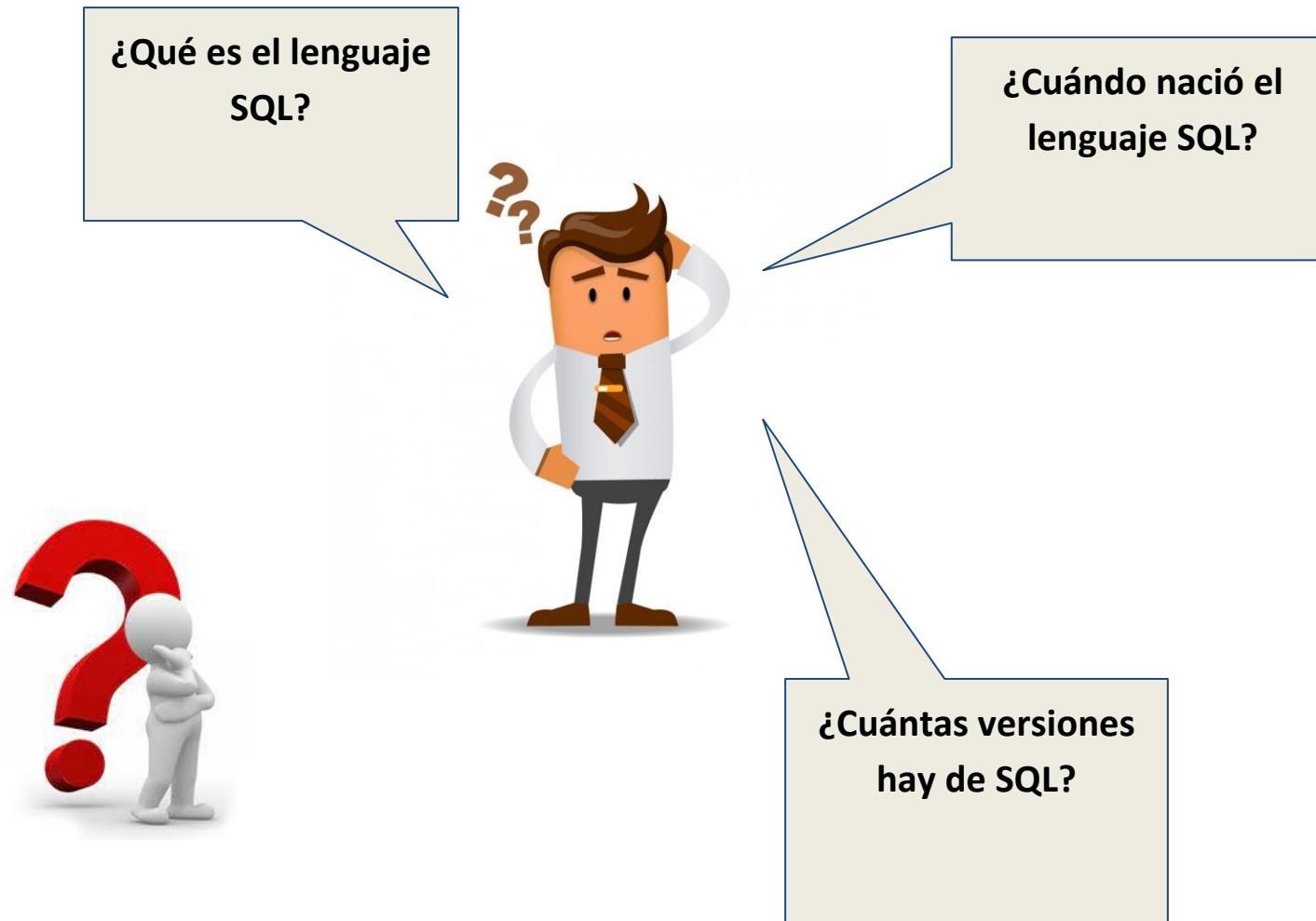


Tema 5. Introducción a SQL. Consultas

1. Introducción
2. Elementos del lenguaje SQL
3. Sintaxis utilizada
4. Tipos de datos
5. Consulta de datos: **SELECT**
 - 5.1. Cláusula FROM
 - 5.2. Cláusula SELECT
 - 5.3. Cláusula WHERE
 - 5.4. Cláusula ORDER BY
 - 5.5. Cláusula LIMIT
6. Combinación de tablas (consultas multitable)
7. Subconsultas
8. Anexos
 - 8.1. Ejemplos de combinación de tablas
 - 8.2. Tablas de los ejercicios



1. Introducción



1. Introducción

¿Qué es el lenguaje SQL?

Es uno de los lenguajes más utilizados de la historia de la informática. Es, sin duda, el **lenguaje fundamental para manejar una base de datos relacional** y, por tanto, es el lenguaje fundamental de los SGBD relacionales.

SQL es un lenguaje declarativo en lo que lo importante es definir qué se desea hacer, por encima de cómo hacerlo (que es la forma de trabajar de los lenguajes de programación de aplicaciones como C o Java). Con este lenguaje se pretendía que las instrucciones se pudieran escribir como si fueran órdenes humanas; es decir, utilizar un lenguaje lo más natural posible. De ahí que se le considere un **lenguaje de cuarta generación**.

Intenta agrupar todas las funciones que se le pueden pedir a una base de datos.

¿Cuándo nació el lenguaje SQL?

¿Cuántas versiones hay de SQL?



1. Introducción

¿Qué es el lenguaje SQL?

¿Cuándo nació el lenguaje SQL?

Data de 1970 cuando E. F. Codd publica su libro: "*Un modelo de datos relacional para grandes bancos de datos compartidos*". Ese libro dictaría las directrices de las bases de datos relacionales. Apenas dos años después IBM (para quien trabajaba Codd) utilizó las directrices de Codd para crear el **Standard English Query Language** (Lenguaje Estándar Inglés para Consultas) al que se le llamó **SEQUEL**. Más adelante se le asignaron las siglas **SQL** (**Standard Query Language**, lenguaje estándar de consulta) aunque en inglés se siguen pronunciando *secuel*.

En 1979 Oracle presenta la primera implementación comercial del lenguaje. Poco después se convertía en un **estándar en el mundo de las bases de datos avalado por los organismos ISO y ANSI**. En el año 1986 se toma como lenguaje estándar por ANSI de los SGBD relacionales. Un año después lo adopta ISO, lo que convierte a SQL en **estándar mundial como lenguaje de bases de datos relacionales**.

¿Cuántas versiones hay de SQL?

1. Introducción

¿Qué es el lenguaje SQL?

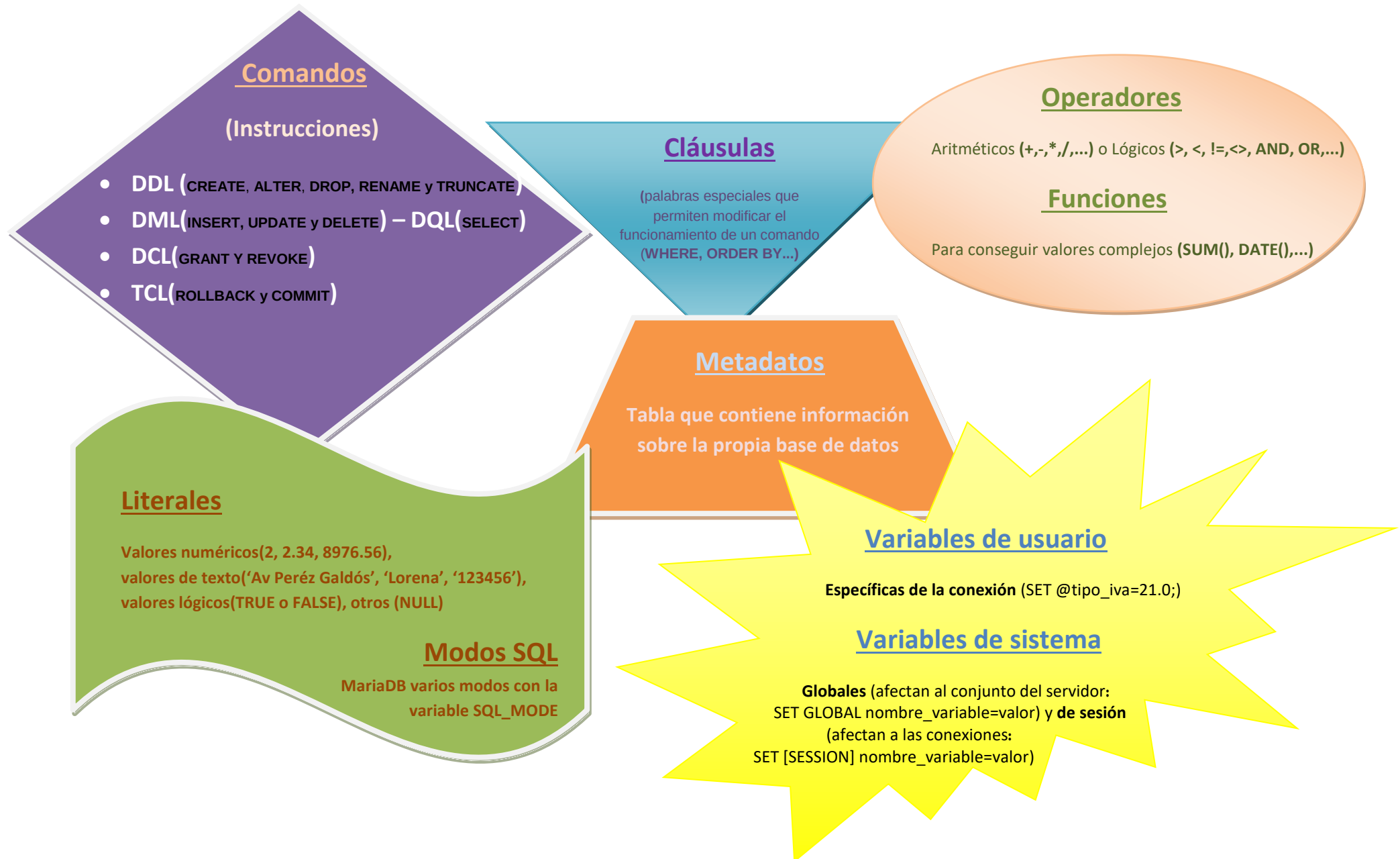
¿Cuándo nació el lenguaje SQL?



En 1989 aparece el estándar ISO (y ANSI) llamado SQL89 o **SQL1**. En 1992 aparece la nueva versión estándar de SQL (a día de hoy sigue siendo la más conocida) llamada **SQL92**. En 1999 se aprueba un nuevo SQL estándar que incorpora mejoras que incluyen triggers, procedimientos, funciones... y otras características de las bases de datos objeto-relacionales; dicho estándar se conoce como **SQL99**. Desde entonces, se han sucedido diferentes versiones (2003, 2006, 2008, 2011) hasta llegar a la última, del año 2016 (**SQL2016**).

¿Cuántas versiones hay de SQL?

2. Elementos del lenguaje SQL



3. Sintaxis utilizada

Sintaxis para **indicar la forma general de un comando** indicando todas las posibles opciones. *Ejemplo:*

```
SELECT * | {[ALL|DISTINCT] columna | expresión [alias],...} FROM tabla;
```

Para indicar la sintaxis de un comando se usan **símbolos** especiales:

-  **PALABRA:** Si se utiliza una palabra en mayúsculas significa que es una palabra que **hay que escribir literalmente**. Esta palabra se podrá escribir en minúsculas también.
-  **Texto:** El texto que aparece en minúsculas sirve para indicar que habrá que **sustituirlo por un objeto de ese tipo**. (*Ejemplo:* SELECT columna FROM tabla;) En el ejemplo, el texto *columna* hay que cambiarlo por un nombre concreto de columna (nombre, apellidos...), al igual que *tabla* se refiere a un nombre de tabla concreto.
-  **[] (corchetes):** Encierran el texto que **no es obligatorio** en el comando, es decir, indican que es opcional ([alias]).
-  **| (barra vertical):** Indica opción. **Se debe elegir** entre esas palabras (ALL|DISTINCT -> o ponemos ALL o DISTINCT).
-  **... (puntos suspensivos):** Indica que **se puede repetir** lo anterior.
-  **{ } (llaves):** Opciones de las que sólo se puede elegir una opción, pero de las que **es obligado elegir una**.

CONSIDERACIONES IMPORTANTES

- ✓ En SQL **no se distingue entre mayúsculas y minúsculas** (las BD y las tablas se corresponden a carpetas y archivos, por lo que los nombres de bases de datos y tablas tienen en cuenta mayúsculas y minúsculas si el SO también las tiene en cuenta para los nombres de archivo. En Linux, sí distingue; en Windows, irrelevante).
- ✓ Las instrucciones **finalizan** con el signo de **punto y coma (;)**
- ✓ A la hora de nombrar objetos es recomendable utilizar **sólo caracteres sin acentos, cifras y los signos \$ y _**.
- ✓ Los comentarios multilínea comienzan por **/*** y terminan por ***/**. Los comentarios de una única línea empiezan con los símbolos **#** (#Esto es un comentario) ó **-- seguido de un espacio en blanco** (-- Esto es un comentario)

4. Tipos de datos

A la hora de crear tablas, **hay que indicar el tipo de datos de cada campo**. Cada SGBD ha creado tipos específicos para necesidades precisas. Todos los tipos de datos que maneja MariaDB están en su **documentación oficial** (<https://mariadb.com/kb/en/data-types/>). Los más destacados:

TIPO	DESCRIPCIÓN
Texto	
CHAR [(M)]	Cadenas de caracteres de longitud fija M (hasta 255). Por defecto 1. Cuando se almacena, la cadena se completa a la derecha con espacios hasta la longitud definida; estos espacios son automáticamente eliminados cuando la cadena se lee.
VARCHAR (M)	Cadenas de caracteres de longitud variable, máximo M (hasta 65535). Cuando se almacena, no se añade ningún espacio extra.
TINYTEXT	Columna de texto con un máximo de 255 (2^8-1) caracteres. Equivalente a VARCHAR(255).
TEXT [(M)]	Columna de texto con un máximo de 65535 ($2^{16}-1$) caracteres, pudiéndose especificar el tamaño M que se desee.
MEDIUMTEXT	Columna de texto con un máximo de 16.777.215 ($2^{24}-1$) caracteres.
LONGTEXT	Columna de texto con un máximo de 4.294.967.295 ($2^{32}-1$) caracteres.
ENUM ('valor1', 'valor2', ...)	Enumeración. Cadena que puede seleccionar un único valor, elegido de la lista de valores que se adjunta. Internamente, los valores ENUM son representado como enteros.
SET ('valor1', 'valor2', ...)	Conjunto. Cadena que puede seleccionar cero o más valores, cada uno de los cuales son escogidos de la lista de valores que se adjunta. Internamente, los valores SET son representados como enteros.

4. Tipos de datos

Números																			
TINYINT	Entero muy pequeño. Ocupa 1 byte. Valores, de -128 a 127 (si es UNSIGNED, de 0 a 255).																		
SMALLINT	Entero pequeño. Ocupa 2 bytes. Valores, de -32768 a 32767 (si es UNSIGNED, de 0 a 65535).																		
MEDIUMINT	Entero mediano. Ocupa 3 bytes. Valores, de -8388608 a 8388607 (si es UNSIGNED, de 0 a 16777215).																		
INT - INTEGER	Entero normal. Ocupa 4 bytes. Valores, de -2.147.483.648 a 2.147.483.647 (si es UNSIGNED, de 0 a 4.294.967.295).																		
BIGINT	Entero largo. Ocupa 8 bytes.																		
DECIMAL [(M [,D])] DEC, NUMERIC y FIXED son sinónimos a DECIMAL A partir de MariaDB 10.3, NUMBER también es sinónimo a DECIMAL	Número en punto fijo (valores exactos). Utilizados cuando es necesario guardar los valores exactos sin redondeos (por ejemplo, para datos monetarios). M indica el número de dígitos en total (la precisión), con un rango de 1 a 65. D indica el número de cifras decimales, con un rango de 0 a 38 (desde MariaDB 10.2.1; antes a 30). Si M se omite, por defecto se toma el valor 10. Si D se omite, por defecto se toma el valor 0. Ejemplo: <table><tr><th>Formato</th><th>Número escrito por el usuario</th><th>Se almacena como...</th></tr><tr><td>DECIMAL</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (9)</td><td>7,456,123.89</td><td>7456124</td></tr><tr><td>DECIMAL (9,2)</td><td>7,456,123.89</td><td>7456123.89</td></tr><tr><td>DECIMAL (6)</td><td>7,456,123.89</td><td>Da error de precisión</td></tr><tr><td>DECIMAL (7,-2)</td><td>7,456,123.89</td><td>7456100</td></tr></table>	Formato	Número escrito por el usuario	Se almacena como...	DECIMAL	7,456,123.89	7456123.89	DECIMAL (9)	7,456,123.89	7456124	DECIMAL (9,2)	7,456,123.89	7456123.89	DECIMAL (6)	7,456,123.89	Da error de precisión	DECIMAL (7,-2)	7,456,123.89	7456100
Formato	Número escrito por el usuario	Se almacena como...																	
DECIMAL	7,456,123.89	7456123.89																	
DECIMAL (9)	7,456,123.89	7456124																	
DECIMAL (9,2)	7,456,123.89	7456123.89																	
DECIMAL (6)	7,456,123.89	Da error de precisión																	
DECIMAL (7,-2)	7,456,123.89	7456100																	

4. Tipos de datos

Números	
FLOAT [(M,D)]	Número en punto flotante (valores aproximados) de 4 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales. Si se omiten M y D, los valores son almacenados a los límites permitidos por el hardware.
DOUBLE [(M,D)] REAL es sinónimo a DOUBLE	Número en punto flotante de 8 bytes. M indica el número de dígitos en total (precisión) y D es el número de cifras decimales.
BIT [(M)]	Campo tipo bit. M indica el número de bits por valor, de 1 a 64. Si M se omite, por defecto es 1.
La mayoría de los tipos numéricos pueden utilizar los modificadores SIGNED (con signo, por defecto), UNSIGNED (sin signo) o ZEROFILL (rellena con ceros por la izquierda).	
Booleanos	
BOOL – BOOLEAN	Equivalente a TINYINT(1). Un valor de cero es considerado FALSE. Un valor no cero es considerado TRUE.
Fechas	
DATE	Fecha. Formato por defecto, 'YYYY-MM-DD'; otros, 'YY-MM-DD', 'YYMMDD', 'YYYY/MM/DD'...
TIME [(<precisión microsecs>)]	Hora. El rango es de '-838:59:59.999999' a '838:59:59.999999'. El formato en el que se muestra por defecto es 'HH:MM:SS.ssssss'; otros, 'D HH:MM:SS', 'HH:MM:SS', 'HH:MM', 'D HH:MM', 'D HH', 'SS', or 'HHMMSS'...

4. Tipos de datos

Fechas	
DATETIME	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.ffffff'. Desde MariaDB 10.3, DATE con una porción de tiempo es sinónimo del tipo DATETIME. Rango posible desde 1000-01-01 00:00:00.000000 a 9999-12-31 23:59:59.999999.
TIMESTAMP	Fecha y hora en formato 'YYYY-MM-DD HH:MM:SS.ffffff'. Este campo se usa generalmente para definir en qué momento se agregó o actualizó una fila y, de manera predeterminada, se le asignará automáticamente la fecha y hora actual cuando se inserte o actualice un registro. Rango posible desde '1970-01-01 00:00:01' a '2038-01-19 03:14:07'.
YEAR [(4)]	Año en formato de 2 ó 4 dígitos (por defecto, 4). El formato de 2 dígitos se considera obsoleto. En el formato de 4 dígitos, el rango permitido es desde 1901 a 2155.
Para almacenar la fecha actual, la mayoría de los SGBD proporcionan funciones, como SYSDATE(), que devuelven ese valor.	
Datos de gran tamaño	
BLOB [(M)]	Objeto binario de un máximo de 65535 ($2^{16}-1$) bytes. Cada valor BLOB se almacena utilizando un prefijo de 2 bytes de longitud que indica el número de bytes en el valor. Si se añade el valor M, MariaDB crea una columna con un tipo BLOB lo suficientemente grande como para almacenar valores de M bytes de longitud.
MEDIUMBLOB	Objeto binario de un máximo de $2^{16}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 3 bytes de longitud que indica el número de bytes en el valor.
LONGBLOB	Objeto binario de un máximo de $2^{32}-1$ bytes. Cada valor BLOB se almacena utilizando un prefijo de 4 bytes de longitud que indica el número de bytes en el valor.

5. Consulta de datos. SELECT

SELECT

Es el comando que se usa para **realizar consultas**. Permite:

- ✚ Seleccionar ciertas columnas de una tabla
- ✚ Seleccionar ciertas filas de acuerdo con ciertos criterios
- ✚ Mezclar datos de tablas diferentes
- ✚ Realizar cálculos sobre los datos
- ✚ Agrupar los datos y realizar cálculos sobre los grupos
- ✚ Realizar consultas complejas, con muchas tablas, pero **¡cuidado!** ¡hay que poner atención al rendimiento que puede disminuir en un comando SQL mal construido!

Formato general de consulta en MariaDB

```
SELECT
  [ALL | DISTINCT | DISTINCTROW]
  [HIGH_PRIORITY]
  [STRAIGHT_JOIN]
  [SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
  [SQL_CACHE | SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
  select_expr [, select_expr ...]
  [ FROM table_references
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
    [HAVING where_condition]
    [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
    procedure|[PROCEDURE procedure_name(argument_list)]
  [INTO OUTFILE 'file_name' [CHARACTER SET charset_name] [export_options]
```

5. Consulta de datos. SELECT

SELECT

Formato reducido de consulta en MariaDB

```
SELECT
  [DISTINCT] select_expr [, select_expr ...]
  [ FROM table_references
  [WHERE where_condition]
  [GROUP BY {col_name | expr | position} [ASC | DESC], ... [WITH ROLLUP]]
  [HAVING where_condition]
  [ORDER BY {col_name | expr | position} [ASC | DESC], ...]
  [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

¡El resultado de una consulta siempre será una tabla de datos, que puede tener una o varias columnas y ninguna, una o varias filas!

¡¡Cuidado con el orden en que se ejecuta cada una de las cláusulas que forman la sentencia SELECT!!

Cláusula	Obligatoria / Opcional	Expresión
FROM	Obligatoria	Tabla(s) fuente(s)
WHERE	Opcional	Condición (condiciones) o restricción (restricciones)
GROUP BY	Opcional	Agrupación (agrupaciones)
HAVING	Opcional	Condición (condiciones) o restricción (restricciones) sobre la(s) agrupación (agrupaciones)
SELECT	Obligatoria	Lista columna(s) y/o elementos de extracción
ORDER BY	Opcional	Ordenación (ordenaciones)
LIMIT	Opcional	Limitar el número de filas devueltas en la consulta

5.1. Cláusula FROM

Permite indicar la tabla o tablas de las que se sacarán los datos. Si hay más de una tabla se realiza el producto cartesiano de las tablas, es decir, se combinan todas las filas de una tabla con todas las de la(s) otra(s) tabla(s). Si se desea, detrás del nombre de tabla se puede poner un alias y usarlo dentro de la consulta.

5.2. Cláusula SELECT

Tomamos de ejemplo la tabla alumno:

id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	telefono
1	María	Sánchez	Pérez	1990-12-01	no	(NULL)
2	Juan	Sáez	Vega	1998-04-02	no	618253876
3	Pepe	Ramírez	Gea	1988-01-03	no	(NULL)
4	Lucía	Sánchez	Ortega	1993-06-13	sí	678516294
5	Paco	Martínez	López	1995-11-24	no	692735409
6	Irene	Gutiérrez	Sánchez	1991-03-28	sí	(NULL)
7	Cristina	Fernández	Ramírez	1996-09-17	no	628349590
8	Antonio	Carretero	Ortega	1994-05-20	sí	612345633
9	Manuel	Domínguez	Hernández	1999-07-08	no	(NULL)
10	Daniel	Moreno	Ruiz	1998-02-03	no	(NULL)

La **cláusula SELECT** permite indicar cuáles serán las columnas que tendrá la tabla de resultado de la consulta que se está realizando. Las opciones que se pueden indicar son:

- El **nombre de una o varias columnas** (separadas por coma) de la tabla sobre la que estamos realizando la consulta. Por ejemplo **SELECT nombre** ó **SELECT id, nombre, es_repetidor**
- Una **constante** que aparecerá en todas las filas de la tabla resultado. Por ejemplo **SELECT '5'** o **SELECT 'estudiante'**
- Una **expresión que nos permite calcular** nuevos valores. Por ejemplo **SELECT concat(nombre, ' ', apellido1)**

5.2. Cláusula SELECT

Las particularidades más destacadas de esta cláusula son los siguientes:

*** (asterisco).** Significa que se seleccionan todas las columnas de las tablas elegidas. Por ejemplo, la consulta:

SELECT * FROM alumno devolverá todo de la tabla alumno (lo que está en la diapositiva anterior).

Sin embargo, si hago la consulta **SELECT nombre, apellido1 FROM alumno** devolverá lo siguiente:

nombre	apellido1
María	Sánchez
Juan	Sáez
Pepe	Ramírez
Lucía	Sánchez
Paco	Martínez
Irene	Gutiérrez
Cristina	Fernández
Antonio	Carretero
Manuel	Domínguez
Daniel	Moreno

Select_expr. Es el **nombre de una columna** (o una **expresión** formada a partir de las columnas) de las tablas que se desea mostrar en la salida del SELECT (en el caso de la consulta anterior sería nombre y apellido1).

ALL. Se muestran **todas las filas de la salida, aunque estén duplicadas** (es la opción por defecto).

Es lo mismo que si no ponemos nada detrás del SELECT, es decir SELECT ALL es igual a solo SELECT

DISTINCT. Hace que **no se muestren las filas duplicadas** en la salida.

13	SELECT DISTINCT es_repetidor FROM alumno;
alumno (2r x 1c)	
es_repetidor	
no	
sí	

5.2. Cláusula SELECT

alias. En una consulta SQL que contenga varias tablas, **es preferible asignar un diminutivo (alias) a cada nombre de tabla**, siempre bajo la óptica de hacer que las consultas sean más legibles para todos los programadores. Este alias se puede utilizar para un nombre de una columna y también para un resultado de una función o un SELECT anidado. Debe escribirse tal cual o entre comillas simples si contiene blancos. La palabra clave AS puede omitirse. Se pueden usar en los SELECT y en los ORDER BY.

```
15 SELECT CONCAT(nombre, ' ', apellido1) AS Nombre FROM alumno;
```

alumno (10r x 1c)	
Nombre	
Maria Sánchez	
Juan Sáez	
Pepe Ramírez	
Lucía Sánchez	
Paco Martínez	
Irene Gutiérrez	
Cristina Fernández	
Antonio Carretero	
Manuel Domínguez	
Daniel Moreno	

Expresiones numéricas. Los operadores + (suma), - (resta), * (multiplicación), / (división) y % (módulo – devuelve el resto entero de una división), se pueden utilizar para hacer **cálculos numéricos en las consultas**.

Más prioridad la multiplicación y división
 menos prioridad la suma y la resta
 En caso de igualdad de prioridad,
 se realiza primero la operación que esté
 más a la izquierda.

```
17 SELECT telefono, telefono*6 FROM alumno;
```

alumno (10r x 2c)	
telefono	telefono*6
(NULL)	(NULL)
618253876	3.709.523.256
(NULL)	(NULL)
678516294	4.071.097.764
692735409	4.156.412.454

Se puede alterar esa prioridad
 usando paréntesis,
 ejecutándose primero lo que
 esté en el interior de los
 paréntesis.

Cuando una expresión aritmética se calcula sobre valores *NULL*, el resultado es el propio valor *NULL*.

Funciones. Es posible hacer uso de funciones específicas de SQL en la cláusula SELECT. Por ejemplo, la función CONCAT se utiliza para concatenar dos cadenas.

5.3. Cláusula WHERE

Sirve para **restringir los datos de salida de las tablas dando una condición que han de cumplir todos los registros de la salida.**

Se pueden diferenciar varios tipos de condiciones que pueden aparecer en la cláusula WHERE:

- Condiciones para comparar valores o expresiones. (**Operadores aritméticos, de comparación y lógicos**)
- Condiciones para comprobar si un valor está dentro de un rango de valores. (**Operador BETWEEN**)
- Condiciones para comprobar si un valor está dentro de un conjunto de valores. (**Operador IN**)
- Condiciones para comparar cadenas con patrones. (**Operador LIKE**)
- Condiciones para comprobar si una columna tiene valores a NULL. (**Operador IS NULL**)

Los **operandos usados en las condiciones pueden ser nombres de columnas, constantes o expresiones.** Los operadores que podemos usar en las condiciones pueden ser aritméticos, de comparación, lógicos, etc. La lista completa de operadores que se pueden utilizar en MariaDB se puede consultar en <https://mariadb.com/kb/en/operators/>.

Operadores aritméticos

Operador	Significado
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo

Operadores de comparación

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
=	Igual que
<>	Distinto
!=	Distinto

Devuelve todo de aquellos alumnos que tienen por nombre Juan. En este caso, sólo hay uno

```
19 SELECT * FROM alumno WHERE nombre="Juan";
```

alumno (1r x 7c)						
id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	telefono
2	Juan	Sáez	Vega	1998-04-02	no	618253876

5.3. Cláusula WHERE

Operadores lógicos

Operador	Significado	Descripción
AND	Y lógica	Devuelve verdadero si las expresiones a su izquierda y derecha son ambas verdaderas
&&	Y lógica	
OR	O lógica	Devuelve verdadero si cualquiera de las dos expresiones a izquierda y derecha del OR, son verdaderas
	O lógica	
NOT	Negación lógica	Invierte la lógica de la expresión que está a su derecha. Si era verdadera, mediante NOT pasa a ser falso y viceversa
!	Negación lógica	

Devuelve el nombre y apellido1 de los alumnos que tengan de apellido2 'Gea' o que sean repetidores. Es decir, devuelve cualquiera que cumpla una de las dos cosas. Pepe cumple lo de 'Gea' y Lucía, Irena y Antonio son repetidores.

```

21 SELECT nombre, apellido1 FROM alumno
22 WHERE apellido2='Gea' OR NOT es_repetidor='no';

```

alumno (4r x 2c)	
nombre	apellido1
Pepe	Ramírez
Lucía	Sánchez
Irene	Gutiérrez
Antonio	Carretero

Tanto los operadores lógicos, de comparación y aritméticos se pueden utilizar tanto **para comparar números como para comparar textos y fechas**. En el caso de los textos, las comparaciones se hacen en orden alfabético (orden alfabético estricto, es el orden del código ASCII).

***HACER EJERCICIOS DEL 1 AL 12**

Por ejemplo, la siguiente consulta devuelve los id de los alumnos que han nacido posteriormente al 27 de diciembre de 1995:

```

23 SELECT id AS alumno, fecha_nacimiento FROM alumno WHERE fecha_nacimiento>'1995-12-27';

```

alumno (4r x 2c)	
alumno	fecha_nacimiento
2	1998-04-02
7	1996-09-17
9	1999-07-08
10	1998-02-03

5.3. Cláusula WHERE

Operador BETWEEN

Nos permite **obtener datos que se encuentren en un rango**. Los valores inicial y final están incluidos. La sintaxis es la siguiente:

Expresion [NOT] BETWEEN valor_inicial AND valor_final;

Por ejemplo, devolver el id, nombre y apellido1 de los alumnos cuyo nombre esté entre 'Ana' y 'Ernesto' siguiendo el orden del código ASCII:

27 `SELECT id, nombre, apellido1 FROM alumno WHERE nombre BETWEEN 'Ana' AND 'Ernesto';`

alumno (3r x 3c)		
id	nombre	apellido1
7	Cristina	Fernández
8	Antonio	Carretero
10	Daniel	Moreno

Operador IN

Para un conjunto de valores posibles, **permite obtener registros cuyos valores estén en una lista de valores**. Lógicamente, también puede comprobar si el valor de una determinada columna está incluido en una lista de valores. La sintaxis es la siguiente:

Expresion [NOT] IN (lista de valores separados por ,);

Por ejemplo, devolver el id y el nombre de los alumnos cuyo apellido2 sea o 'López', 'García', 'Fernández' o 'Ruiz':

29 `SELECT id, nombre FROM alumno WHERE apellido2 IN('López', 'García', 'Fernández', 'Ruiz');`

alumno (2r x 2c)	
id	nombre
5	Paco
10	Daniel

5.3. Cláusula WHERE

Operador LIKE

Se utiliza **para comparar si una cadena de caracteres coincide con un patrón**. En el patrón podemos utilizar cualquier carácter alfanumérico, pero hay dos caracteres que tienen un significado especial, el símbolo del porcentaje (%) y el carácter de subrayado (_).

- % → Equivale a **una serie cualquiera de caracteres** (puede ser 0 caracteres).
- _ → Equivale a **un carácter cualquiera** (siempre un carácter en su lugar).

En los patrones de comparación, **hay distinción entre mayúsculas y minúsculas**.

Expresión [NOT] LIKE patrón;

Operador REGEXP

El operador REGEXP permite **realizar búsquedas mucho más potentes que LIKE** haciendo uso de expresiones regulares. Ver <https://mariadb.com/kb/en/regular-expressions-overview/> y <https://mariadb.com/kb/en/regexp/>

expr REGEXP pat

Devuelve 1 si *expr* coincide con *pat* o 0 si no coincide. Se puede incluir en el SELECT directamente devolviendo en este caso 0 ó 1.

Por ejemplo, en la primera consulta devuelve el nombre de los alumnos que empiezan y contienen cualquier tipo y número de caracteres y terminan por la 'i' y tras ésta, cualquier letra. En la segunda, los nombres que contienen el patrón 'an' (Ej: hacer **SELECT nombre REGEXP 'an'**):

31 SELECT nombre FROM alumno WHERE nombre LIKE '%i_';

alumno (3r x 1c)
nombre
María
Lucía
Antonio

33 SELECT nombre FROM alumno WHERE nombre REGEXP 'an';

alumno (4r x 1c)
nombre
Juan
Antonio
Manuel
Daniel

5.3. Cláusula WHERE

Operador IS NULL

Devuelve verdadero si el valor que examina es nulo:

Expresión IS [NOT] NULL

37 `SELECT * FROM alumno WHERE telefono IS NOT NULL;`

alumno (5r x 7c)

id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	telefono
2	Juan	Sáez	Vega	1998-04-02	no	618253876
4	Lucía	Sánchez	Ortega	1993-06-13	sí	678516294
5	Paco	Martínez	López	1995-11-24	no	692735409
7	Cristina	Fernández	Ramírez	1996-09-17	no	628349590
8	Antonio	Carretero	Ortega	1994-05-20	sí	612345633

Prioridad en las operaciones

Orden de precedencia	Operador
1	*(multiplicar) / (dividir) % (módulo)
2	- (resta) + (suma)
3	Comparaciones (>, <, !=, ...)
4	IS NULL, LIKE, REGEXP, IN
5	BETWEEN
6	NOT
7	AND
8	OR
9	= (asignación)

5.4. Cláusula ORDER BY

El orden inicial de los registros obtenidos por un SELECT no guarda más que una relación respecto al orden en el que fueron introducidos. **Para ordenar en base a otros criterios se utiliza la cláusula ORDER BY.**

Se puede ordenar sobre cualquier columna de una tabla; para ello, es necesario que las columnas formen parte de la selección, pero no es obligatorio que se muestren.

La sintaxis es:

ORDER BY {col_name | expr | position} [ASC | DESC], ...

En esa cláusula se coloca una lista de campos que indica la forma de ordenar (**el nombre, una expresión o la posición respecto al select**). Se ordena primero por el primer campo de la lista; si hay coincidencias, por el segundo; si ahí también las hay, por el tercero, y así sucesivamente.

Se pueden colocar las palabras **ASC** O **DESC** en cada columna de ordenación para indicar si se desea orden ascendente o descendente (**por defecto se toma ASC – del más pequeño al más grande**; en fechas, de fecha más lejana a más cercana).

```
39 SELECT nombre , es_repetidor FROM alumno
40 ORDER BY es_repetidor, nombre;
```

alumno (10r x 2c)	
nombre	es_repetidor
Cristina	no
Daniel	no
Juan	no
Manuel	no
María	no
Paco	no
Pepe	no
Antonio	sí
Irene	sí
Lucía	sí

```
39 SELECT nombre, fecha_nacimiento FROM alumno
40 ORDER BY fecha_nacimiento DESC;
```

alumno (10r x 2c)	
nombre	fecha_nacimiento
Manuel	1999-07-08
Juan	1998-04-02
Daniel	1998-02-03
Cristina	1996-09-17
Paco	1995-11-24
Antonio	1994-05-20
Lucía	1993-06-13
Irene	1991-03-28
María	1990-12-01
Pepe	1988-01-03

¡¡La ordenación puede ser origen de un bajo rendimiento de la consulta!!

5.5. Cláusula LIMIT

La cláusula LIMIT permite limitar el número de filas que se incluyen en el resultado de la consulta. La sintaxis es:

```
[LIMIT {[offset,] row_COUNT | row_COUNT OFFSET offset}]
```

row_COUNT es el número de filas que queremos obtener y **offset** el número de filas que nos saltamos antes de empezar a contar. Es decir, la primera fila que se obtiene como resultado es la que está situada en la posición *offset+1*.

Por ejemplo, si quiero obtener las **tres filas siguientes a la quinta fila de alumno** haría lo siguiente:

41 `SELECT * FROM alumno LIMIT 5,3;`

alumno (3r × 7c)							
id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	telefono	
6	Irene	Gutiérrez	Sánchez	1991-03-28	sí	(NULL)	
7	Cristina	Fernández	Ramírez	1996-09-17	no	628349590	
8	Antonio	Carretero	Ortega	1994-05-20	sí	612345633	

Esto de aquí es equivalente a lo siguiente:

41 `SELECT * FROM alumno LIMIT 3 OFFSET 5;`

alumno (3r × 7c)							
id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	telefono	
6	Irene	Gutiérrez	Sánchez	1991-03-28	sí	(NULL)	
7	Cristina	Fernández	Ramírez	1996-09-17	no	628349590	
8	Antonio	Carretero	Ortega	1994-05-20	sí	612345633	

***HACER EJERCICIOS DEL 13 AL 25**

6. Combinación de tablas (consultas multitabla)

Es más que habitual necesitar en una consulta datos que se encuentran distribuidos en varias tablas. Las bases de datos relacionales se basan en que los datos se distribuyen en tablas que se pueden relacionar mediante un campo. Ese campo es el que permite integrar los datos de las diferentes tablas.

Por ejemplo, si disponemos de una tabla de empleados cuya clave es el *dni* y otra tabla de tareas que se refiere a tareas realizadas por los empleados, es seguro (si el diseño está bien hecho) que en la tabla de tareas aparecerá el dni del empleado para saber qué empleado realizó la tarea.

En el ejemplo anterior, si quiere obtener **una lista de los datos de las tareas y los empleados**, se podría hacer de esta forma:

```
SELECT cod_tarea, descripcion_tarea, dni_empleado, nombre_empleado
FROM tareas, empleados;
```

- ✓ La sintaxis **es correcta** ya que, efectivamente, en el apartado *FROM* se pueden indicar varias tablas separadas por comas. Sin embargo, **esto produce un producto cruzado o producto cartesiano**, es decir, **aparecerán todos los registros de las tareas combinados con todos los registros de empleados**, es decir, devolverá para cada fila de la primera tabla todas las filas del resto de tablas, y así sucesivamente (si tenemos 3 y 2 filas en cada tabla, el sistema devolverá $3 \times 2 = 6$ filas) tal y como se puede apreciar en la siguiente imagen:

Tabla 1	
campo 1	campo 2
a1	b1
a2	b2
a3	b3

Tabla 2	
campo 11	campo 22
a4	b4
a5	b5

Tabla1 X Tabla 2			
campo 1	campo 2	campo 11	campo 22
a1	b1	a4	b4
a1	b1	a5	b5
a2	b2	a4	b4
a2	b2	a5	b5
a3	b3	a4	b4
a3	b3	a5	b5

***HACER EL EJERCICIO 26**

6. Combinación de tablas (consultas multitabla)

JOIN – Combinación o asociación entre tablas

El producto cartesiano a veces es útil para realizar consultas complejas, pero en el caso normal no lo es ya que lo lógico sería “vincular” las tablas entre ellas para obtener un resultado conforme a lo que esperamos y que sea realista, es decir, para este caso particular, **necesitamos filtrar ese producto para que sólo aparezcan los registros de las tareas relacionadas con sus empleados correspondientes**. A eso se le llama combinar o asociar (**JOIN**) tablas.

Para realizar este tipo de consultas podemos usar dos alternativas.

Alternativa 1 – Sintaxis SQL1

La forma de realizar correctamente la consulta anterior asociando las tareas con los empleados que la realizaron sería indicando la relación existente entre las diferentes tablas (a través de la clave foránea) en la cláusula WHERE:

```
SELECT cod_tarea, descripc, dni_emple, nombre
FROM tareas, empleados
WHERE tareas.dni_emple = empleados.dni;
```

EMPLEADOS

dni	nombre	tfno
12345689Y	María	941222832
98765432F	Juan	605666845
72732827K	Sheila	941292928
****	****	****
****	****	****

TAREAS

cod_tarea	descripc	dni_emple
01	Supervisar	12345689Y
02	Diseñar	98765432F
03	Analizar	72732827K
04	Programar	72732827K
05	Arreglar	98765432F

Nótese que se utiliza la notación **tabla.columna** para evitar ambigüedades, ya que el mismo nombre de campo se puede repetir en ambas tablas (si en vez de dni_emple se especificara sólo dni).

Si algunas columnas tienen el mismo nombre perteneciendo a tablas distintas, habrá que añadir el nombre de la tabla delante de la columna, para que el sistema sepa qué columna debe elegir. **No es obligatorio poner el nombre de las tablas delante de cada columna, pero por una cuestión de legibilidad y de mantenimiento, es preferible ponerlas cuando se trate de selecciones complejas.**

6. Combinación de tablas (consultas multitabla)

JOIN – Combinación o asociación entre tablas

Alternativa 1 – Sintaxis SQL1

Para evitar repetir continuamente el nombre de la tabla, se puede utilizar un alias de tabla:

```
SELECT a.cod_tarea, a.descripcion_tarea, b.dni_empleado, b.nombre_empleado
FROM tareas a, empleados b
WHERE a.dni_empleado = b.dni;
```

Al apartado WHERE se le pueden añadir condiciones, encadenándolas con el operador AND.

Ejemplo:

```
SELECT a.cod_tarea, a.descripcion_tarea
FROM tareas a, empleados b
WHERE a.dni_empleado = b.dni AND b.nombre_empleado='Javier';
```

Finalmente, también se pueden enlazar más de dos tablas a través de sus campos relacionados.

Ejemplo:

```
SELECT a.cod_tarea, a.descripcion_tarea, b.nombre_empleado, c.nombre_utensilio
FROM tareas a, empleados b, utensilios_utilizados c
WHERE a.dni_empleado = b.dni AND a.cod_tarea=c.cod_tarea;
```

A las relaciones descritas anteriormente se las llama relaciones en igualdad (**equijoins**), ya que las tablas se relacionan a través de campos que contienen valores iguales en dos tablas. Son las más frecuentes y se suelen llamar **combinaciones internas**, porque sólo aparecen las filas relacionadas.

***HACER EL EJERCICIO 27**

6. Combinación de tablas (consultas multitabla)

JOIN – Combinación o asociación entre tablas

Alternativa 2 – Sintaxis SQL2

En la versión SQL de 1992 se ideó una **nueva sintaxis para consultar varias tablas de forma combinada**. La razón fue separar las condiciones de asociación respecto de las condiciones de selección de registros.

La sintaxis completa es:

```
SELECT tabla1.columna1, tabla1.columna2,... tabla2.columna1, tabla2.columna2,...
      FROM tabla1
      [CROSS JOIN tabla2] |
      [NATURAL JOIN tabla2] |
      [JOIN tabla2 USING (columna)] |
      [[INNER] JOIN tabla2
ON (tabla1.columa=tabla2.columna)] |
      [{LEFT|RIGHT} [OUTER] JOIN tabla2
ON (tabla1.columa=tabla2.columna)]
      [WHERE ....] [ORDER BY ...];
```

Basándonos en la segunda alternativa, vamos a ver las distintas posibilidades (**CROSS JOIN, NATURAL JOIN, JOIN USING, INNER JOIN, OUTER JOIN**). Para ello, nos basamos en estas dos tablas:

- Alumno (DNI, Nom_alumno, Ape_alumno, Direccion, Tfno, Fecha_nac, Id_curso[↑])
- Curso (Id_curso, Nom_curso, Tutor, Aula)

6. Combinación de tablas (consultas multitabla)

CROSS JOIN

Se realiza un **producto cruzado o cartesiano** entre las tablas indicadas. Eso significa que **cada tupla de la primera tabla se combina con cada tupla de la segunda tabla**. Es decir, si la primera tabla tiene 10 filas y la segunda otras 10, como resultado se obtienen 100 filas, resultado de combinar todas entre sí.

Ejemplo: ***SELECT * FROM alumno CROSS JOIN curso;***

No es una operación muy utilizada, aunque posibilita resolver consultas extremadamente complicadas. Es igual que poner los nombres de las dos tablas, sin especificar la asociación.

***HACER EL EJERCICIO 28. COMPRUEBA EL RESULTADO CON EL EJERCICIO 26**

NATURAL JOIN

Establece una **relación de igualdad** entre las tablas a través de los campos que tengan el mismo nombre en ambas tablas.

Ejemplo: ***SELECT * FROM alumno NATURAL JOIN curso;***

En el ejemplo anterior, se obtienen los registros de alumnos relacionados con los cursos a través de los campos que tengan el mismo nombre en ambas tablas (en este caso, `id_curso`). **Hay que asegurarse de que sólo son las claves principales y secundarias de las tablas relacionadas las columnas en las que el nombre coincide**, de otro modo fallaría la asociación y la consulta no funcionaría.

No se aconseja utilizar este tipo de unión, sobre todo por razones de legibilidad. También hay riesgo de que dos columnas tengan el mismo nombre sin pertenecer a una clave, y así el resultado no será el que queremos.

***HACER EL EJERCICIO 29. COMPRUEBA EL RESULTADO CON EL EJERCICIO 27**

6. Combinación de tablas (consultas multitabla)

JOIN USING

Permite **establecer relaciones** indicando qué **columna** (o columnas) **común a las dos tablas** hay que utilizar.

Ejemplo: ***SELECT * FROM alumnos JOIN cursos USING (id_curso);***

Las columnas deben de tener exactamente el mismo nombre en ambas tablas, pero si hay otras con el mismo nombre, no las combinará (si, por ejemplo, tanto en alumnos como en cursos se tiene el campo nombre, habría que usar JOIN USING en lugar de NATURAL JOIN).

***HACER EL EJERCICIO 30. COMPRUEBA EL RESULTADO CON EL EJERCICIO 29**

INNER JOIN

Permite **establecer relaciones** cuya **condición se establece manualmente**, lo que permite realizar asociaciones más complejas, **o bien asociaciones** cuyos campos en las tablas no tienen el mismo nombre.

Ejemplo: ***SELECT * FROM alumnos [INNER] JOIN cursos ON (cursos.id_curso=alumnos.id_curso);***

Utilizando las formas anteriores de relacionar tablas (incluidas las explicadas con la sintaxis SQL 92), sólo aparecen en el resultado de la consulta filas presentes en las tablas relacionadas, es decir, aplicado al ejemplo que tenemos, sólo aparecen alumnos asignados a algún curso. Si hay cursos que no tienen alumnos, esos cursos no aparecen.

***HACER EL EJERCICIO 31. COMPRUEBA EL RESULTADO CON EL EJERCICIO 27**

Lo explicado en este apartado para unir dos tablas es igualmente aplicable si se necesitan unir tres o más tablas (***SELECT *FROM cliente INNER JOIN empleado ON cliente.codigo_empleado_rep_ventas = empleado.codigo_empleado INNER JOIN pago ON cliente.codigo_cliente = pago.codigo_cliente***) o sobre la misma tabla, para lo que sería necesario utilizar un alias para la tabla (***SELECT empleado.nombre, empleado.apellido1, empleado.apellido2, jefe.nombre, jefe.apellido1, jefe.apellido2 FROM empleado INNER JOIN empleado AS jefe ON empleado.codigo_jefe = jefe.codigo_empleado;***) ***HACER EL EJERCICIOS 32 Y 33**

6. Combinación de tablas (consultas multitabla)

OUTER JOIN – Combinaciones externas

Si en una combinación de tablas **se desea incluir alguna fila de una tabla que no esté relacionada con filas de la otra tabla se utilizan relaciones laterales o externas (*outer join*)**, existiendo las diferentes posibilidades:

- **LEFT OUTER JOIN**. Devolverá todas las filas de la tabla que hemos colocado a la izquierda de la composición y relacionará las filas de la tabla de la izquierda con las filas de la tabla de la derecha con las que encuentra una coincidencia. Si no encuentra ninguna coincidencia, se mostrarán los valores de la fila de la tabla izquierda y en los valores de la tabla derecha donde no ha encontrado ninguna coincidencia mostrará el valor NULL.
- **RIGHT OUTER JOIN**. Esta consulta devolverá todas las filas de la tabla que hemos colocado a la derecha de la composición y relacionará las filas de la tabla de la derecha con las filas de la tabla de la izquierda con las que encuentre una coincidencia. Si no encuentra ninguna coincidencia, se mostrarán los valores de la fila de la tabla derecha y en los valores de la tabla izquierda donde no ha encontrado una coincidencia mostrará el valor NULL.
- **FULL OUTER JOIN**. *Esta opción no está implementada como tal en MariaDB.* Sin embargo, se podría simular utilizando el operador UNION (se estudiará en un tema posterior), que nos realiza la unión del resultado de dos consultas. El resultado esperado de una composición FULL OUTER JOIN es obtener la intersección de las dos tablas, junto con las filas de ambas tablas que no se pueden combinar. Dicho con otras palabras, el resultado sería el equivalente a realizar la unión de una consulta de tipo LEFT JOIN y una consulta de tipo RIGHT JOIN sobre las mismas tablas.
- **NATURAL [LEFT | RIGHT] OUTER JOIN**. Esta opción realiza el LEFT ó RIGHT JOIN entre las dos tablas tal y como se ha especificado en los casos anteriores respectivamente, pero, la única diferencia es que en este caso no es necesario utilizar la cláusula ON para indicar sobre qué columna vamos a relacionar las dos tablas. En este caso las tablas se van a relacionar sobre aquellas columnas que tengan el mismo nombre, por lo que hay que tener mucho cuidado al aplicar esta cláusula y asegurarnos bien de que las columnas tienen el mismo nombre.

La sintaxis de los **OUTER JOIN** es:

```
SELECT columnas
FROM tabla1 {LEFT | RIGHT} [OUTER] JOIN tabla2
ON (condición);
```

6. Combinación de tablas (consultas multitabla)

Considerando las diferentes opciones sobre un ejemplo práctico, supongamos que tenemos dos tablas: *empleado* y *departamento*, relacionadas por el *id del departamento*, con los siguientes datos.

Tabla: empleado			Tabla: departamento	
id	nombre	id_departamento	id	nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	3	Recursos Humanos

Si hiciéramos:

```
SELECT * FROM empleado LEFT JOIN departamento  
ON empleado.codigo_departamento = departamento.codigo;
```

El resultado de la operación **LEFT JOIN** es:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	NULL	NULL

Si hiciéramos:

```
SELECT * FROM empleado RIGHT JOIN departamento  
ON empleado.codigo_departamento = departamento.codigo;
```

El resultado de la operación **RIGHT JOIN** es:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
NULL	NULL	NULL	3	Recursos Humanos

6. Combinación de tablas (consultas multitabla)

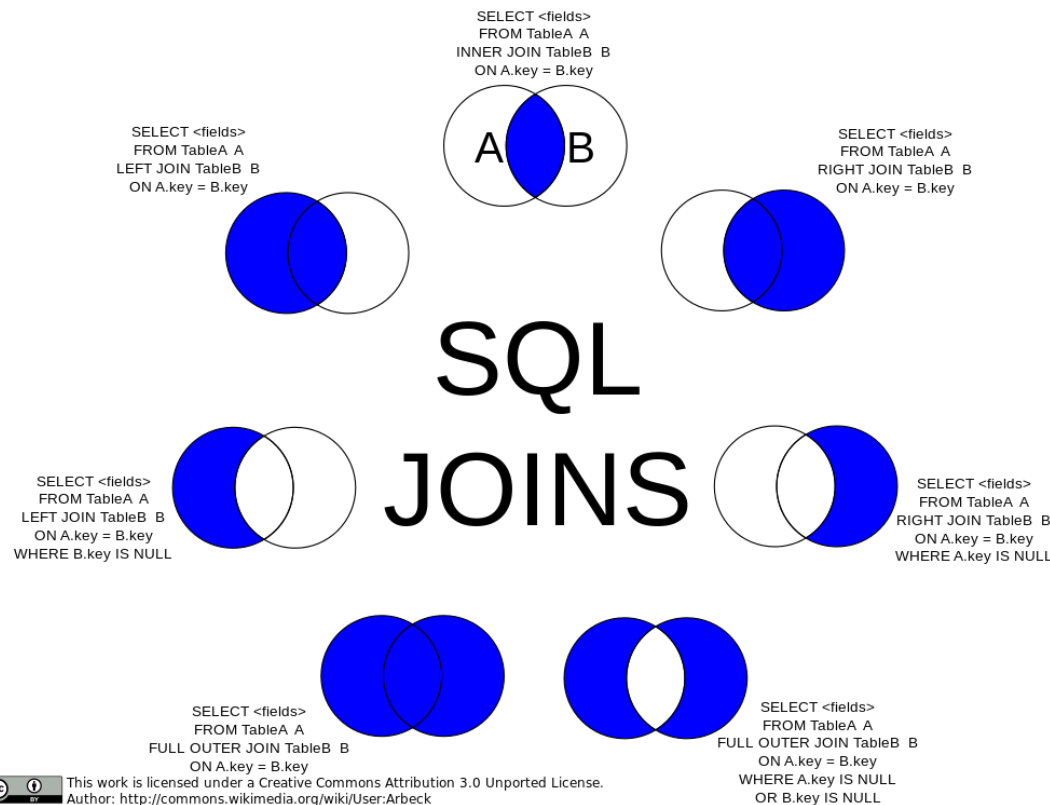
Si hiciéramos:

```
SELECT * FROM empleado LEFT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo
UNION
SELECT FROM empleado RIGHT JOIN departamento
ON empleado.codigo_departamento = departamento.codigo;
```



El resultado de la simulación de la operación FULL JOIN sería:

empleado. id	empleado. nombre	empleado. id_departamento	departamento. id	departamento. nombre
1	Pepe	1	1	Desarrollo
2	María	2	2	Sistemas
3	Juan	NULL	NULL	NULL
NULL	NULL	NULL	3	Recursos Humanos



Si consideramos otro ejemplo, teniendo las tablas *cursos* y *alumnos*, esta consulta:

```
SELECT * FROM cursos
LEFT OUTER JOIN alumnos
ON (cursos.id_curso=alumnos.id_curso);
```

obtiene todas las filas de la tabla *cursos* que estén o no relacionadas con datos de la tabla de *alumnos* (la tabla *LEFT* sería *cursos* porque es la que está a la izquierda del *JOIN*).

En la salida, los *cursos* que no tengan *alumnos* tendrán valores nulos en las columnas de la tabla *alumnos*.

En la consulta anterior, si el *LEFT* lo cambiamos por un *RIGHT*, aparecerán tanto los *alumnos* que tienen *curso* asignado como los que no tienen asignado ningún *curso*.

***HACER DEL EJERCICIO 34 AL 37**



This work is licensed under a Creative Commons Attribution 3.0 Unported License.
Author: <http://commons.wikimedia.org/wiki/User:Arbeck>

7. Subconsultas

Se trata de una técnica que **permite utilizar el resultado de una consulta SELECT en otra consulta SELECT**. **Permite solucionar consultas que requieren para funcionar el resultado previo de otra consulta.**

El estándar SQL define tres tipos de subconsultas:

1. **Subconsultas escalares**. Son aquellas que devuelven una columna y una fila.
2. **Subconsultas de fila**. Son aquellas que devuelven más de una columna, pero una única fila.
3. **Subconsultas de tabla**. Son aquellas que devuelven una o varias columnas y cero o varias filas.

Se puede colocar el SELECT dentro de las cláusulas WHERE, HAVING o FROM. En este tema se estudiarán las subconsultas en la cláusula WHERE.

La sintaxis es:

```
SELECT listaExpresiones FROM tabla
WHERE expresión OPERADOR
(SELECT listaExpresiones
FROM tabla);
```

Los operadores/palabras reservadas que se pueden utilizar en las subconsultas son los siguientes:

- Operadores básicos de comparación: **>, >=, <, <=, !=, <>, =.**
- Predicados **ALL y ANY**.
- Predicado **IN y NOT IN**.
- Predicado **EXISTS y NOT EXISTS**.

Ejemplos: Seleccionar los empleados que cobran menos que GIL.

```
SELECT apellido, salario FROM emple
WHERE salario < (SELECT salario FROM emple WHERE apellido='GIL');
```

7. Subconsultas

Se pueden realizar esas subconsultas las veces que haga falta. Por ejemplo, si queremos saber los que cobran menos que GIL y más que SALA:

```
SELECT apellido, salario
```

```
FROM emple
```

```
WHERE salario < (SELECT salario FROM emple WHERE apellido='GIL')
```

```
AND salario > (SELECT salario FROM emple WHERE apellido='SALA');
```

En realidad, lo primero que hace la base de datos es calcular el resultado de los SELECT más internos, para luego calcular el más externo.

Las subconsultas siempre se deben encerrar entre paréntesis y se debería colocar a la derecha del operador relacional. **Una subconsulta que utilice los valores >, <, >=, ... tiene que devolver un único valor, de otro modo ocurre un error.**

Además, **tienen que tener el mismo tipo de columna para relacionar la subconsulta con la consulta que la utiliza** (no puede ocurrir que la subconsulta tenga dos columnas y ese resultado se compare usando una sola columna en la consulta general).

Pueden ponerse dos columnas en la condición y que la subconsulta devuelva también dos valores.

Ejemplo:

```
SELECT * FROM emple WHERE (oficio, dept_no) =
```

```
(SELECT oficio, dept_no FROM emple WHERE apellido='REY');
```

7. Subconsultas

SUBCONSULTAS CON OPERADORES BÁSICOS DE COMPARACIÓN (>, >=, <, <=, !=, <>, =)

Los operadores básicos de comparación (>, >=, <, <=, !=, <>, =) se pueden usar cuando queremos comparar una expresión con el valor que devuelve una subconsulta.

Los operadores básicos de comparación los vamos a utilizar para realizar comparaciones con subconsultas que devuelven un único valor, es decir, una columna y una fila.

Ejemplo: Devuelve todos los productos que tienen un precio mayor o igual al producto más caro del fabricante Asus.

```
SELECT *  
FROM producto  
WHERE precio >= (SELECT MAX(precio)  
                 FROM fabricante  
                 INNER JOIN producto  
                 ON  
                 fabricante.codigo = producto.codigo_fabricante  
                 WHERE fabricante.nombre = 'Asus');
```



```
SELECT *  
FROM producto  
WHERE precio = (SELECT MAX(precio)  
                FROM producto  
                WHERE codigo_fabricante = (SELECT codigo  
                                           FROM fabricante  
                                           WHERE nombre = 'Asus'));
```

7. Subconsultas

SUBCONSULTAS CON ALL Y ANY

ALL y **ANY** se utilizan con los operadores de comparación (>, >=, <, <=, !=, <>, =) y **nos permiten comparar una expresión con el conjunto de valores que devuelve una subconsulta.**

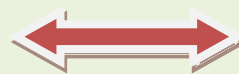
ALL y ANY los vamos a utilizar **para realizar comparaciones con subconsultas que pueden devolver varios valores**, es decir, una columna y varias filas. La palabra reservada **SOME** es un alias de **ANY**, por lo que se puede usar indistintamente.

Ejemplo: Seleccionar el empleado que más cobra:

```
SELECT nombre, sueldo
FROM empleados
WHERE sueldo >= ALL (SELECT sueldo FROM empleados);
```

Ejemplo: De dos maneras diferentes, devuelve todos los productos de la base de datos que tienen un precio mayor o igual al producto más caro del fabricante Asus; una, haciendo uso de operadores de comparación y dos, haciendo uso de ALL.

```
SELECT *
FROM fabricante INNER JOIN producto
ON fabricante.codigo = producto.codigo_fabricante
WHERE precio >= (SELECT MAX(precio)
FROM fabricante
INNER JOIN producto ON
fabricante.codigo = producto.codigo_fabricante
WHERE fabricante.nombre = 'Asus');
```



```
SELECT *
FROM fabricante INNER JOIN producto
ON fabricante.codigo = producto.codigo_fabricante
WHERE precio >= ALL (SELECT precio
FROM fabricante INNER JOIN producto
ON fabricante.codigo = producto.codigo_fabricante
WHERE fabricante.nombre = 'Asus');
```

7. Subconsultas

SUBCONSULTAS CON IN Y NOT IN

IN y **NOT IN** nos permiten **comprobar si un valor está o no incluido en un conjunto de valores**, que puede ser el conjunto de valores que devuelve una subconsulta. IN y NOT IN los vamos a utilizar **para realizar comparaciones con subconsultas que pueden devolver varios valores**, es decir, una columna y varias filas.

Ejemplo: Obtener los nombres de los empleados cuyos dni están en la tabla de directivos:

```
SELECT nombre
FROM empleados
WHERE dni IN (SELECT dni FROM directivos);
```

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
SELECT *
FROM cliente
WHERE id NOT IN (SELECT id_cliente FROM pedido);
```

Cuando se esté trabajando **en subconsultas**, **IN y = ANY** realizan la misma función (igual sucedería con **NOT IN** y **<> ALL**). Por lo tanto, las siguientes consultas devolverían el mismo resultado:

```
SELECT s1 FROM t1 WHERE s1 = ANY (SELECT s1 FROM t2);
```



```
SELECT s1 FROM t1 WHERE s1 IN (SELECT s1 FROM t2);
```

Si se necesita **comprobar dos columnas** en una consulta IN, se haría como en el ejemplo siguiente:

```
SELECT nombre FROM empleados WHERE (cod1, cod2) IN (SELECT cod1, cod2 FROM directivos);
```

Es importante tener presente **que cuando hay un valor NULL en el resultado de la consulta interna, la consulta externa no devuelve ningún valor**.

La forma de solucionarlo sería quitando los valores NULL de la consulta interna, tal y como se muestra a continuación:

```
SELECT nombre FROM departamento WHERE codigo NOT IN (SELECT codigo_departamento
FROM empleado
WHERE codigo_departamento IS NOT NULL);
```

7. Subconsultas

SUBCONSULTAS CON EXISTS Y NOT EXISTS

Subconsultas que utilicen la palabra reservada **EXISTS** devolverán **true** si la subconsulta devuelve alguna fila; si no, devuelve **false**.

Ejemplo: Devuelve un listado de los clientes que no han realizado ningún pedido.

```
SELECT *
FROM cliente
WHERE NOT EXISTS (SELECT id_cliente
                  FROM pedido
                  WHERE cliente.id = pedido.id_cliente);
```

Se utiliza bastante en consultas correlacionadas.

SUBCONSULTAS CORRELACIONADAS

Son aquellas consultas en las que la consulta interna hace referencia a la consulta externa o, dicho de otro modo, ocurren cuando la subconsulta debe hacer referencia a una columna o varias de la consulta externa. Si las tablas usadas en la consulta y la subconsulta coinciden, habrá que usar alias de tabla. Un caso general podría ser:

```
SELECT * FROM t1 WHERE column1 = ANY
(SELECT column1 FROM t2 WHERE t2.column2 = t1.column2);
```

Entender este tipo de consultas puede resultar complicado. En este ejemplo, tenga en cuenta que la subconsulta contiene una referencia a una columna de t1, incluso aunque la cláusula FROM de la subconsulta no menciona una tabla t1. Por lo tanto, el SGBD busca fuera de la subconsulta y encuentra t1 en la consulta externa. Supongamos que la tabla t1 contiene un registro en que column1=5 y column2=6; mientras, la tabla t2 contiene un registro en que column1=5 y column2=7.

La expresión ...WHERE column1 = ANY (SELECT column1 FROM t2) sería TRUE, pero en este ejemplo, la cláusula WHERE dentro de la subconsulta es FALSE (ya que (5,6) no es igual a (5,7)), así que la subconsulta como un todo es FALSE.

7. Subconsultas

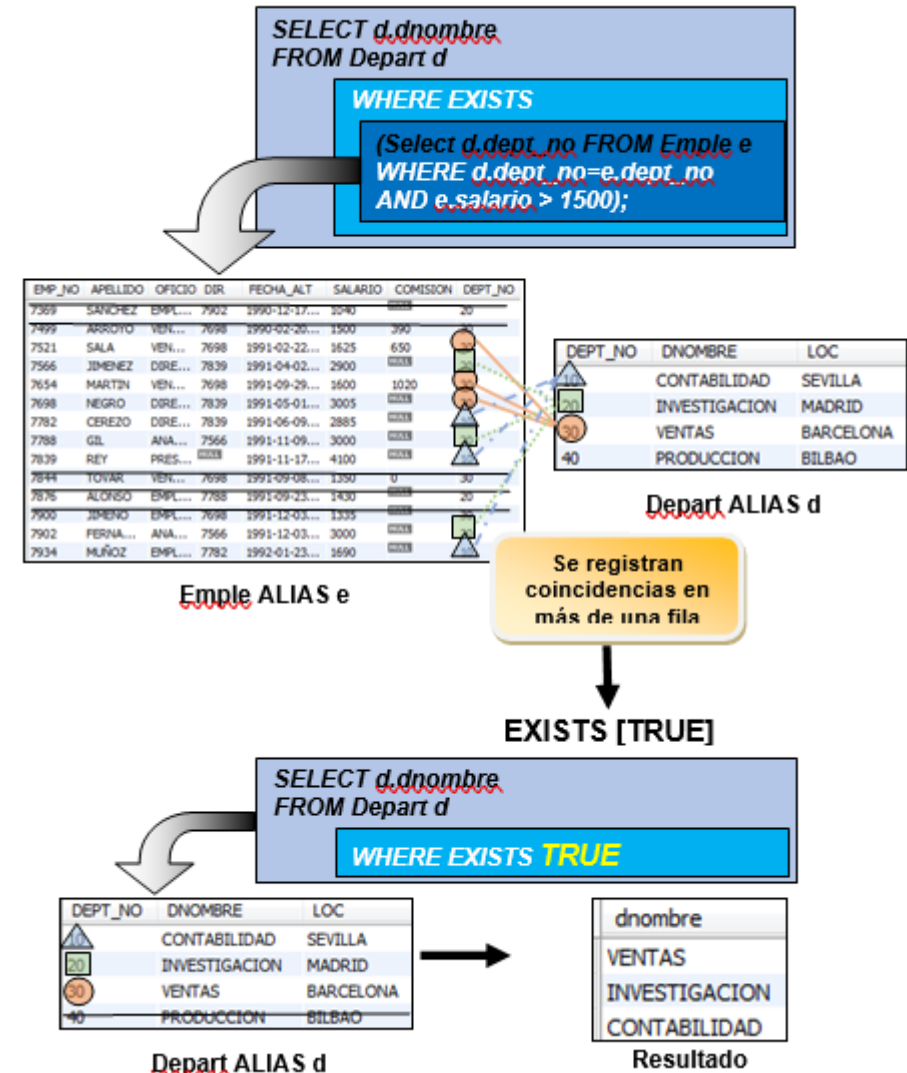
SUBCONSULTAS CON EXISTS Y NOT EXISTS

Ejemplo: Departamentos en los que algún empleado gana más de 1500€

```
SELECT d.dnombre
FROM Depart d
WHERE EXISTS (SELECT d.dept_no
FROM Emple e
WHERE d.dept_no=e.dept_no
AND e.salario>1500);
```

dnombre
VENTAS
INVESTIGACION
CONTABILIDAD

Gráficamente:



7. Subconsultas

SUBCONSULTAS CON EXISTS Y NOT EXISTS

Ejemplo: Mostrar los empleados cuyo salario sea el máximo de su departamento:

```
SELECT apellido, salario
FROM emple e WHERE salario >= ALL
      (SELECT salario FROM emple emp
       WHERE e.dept_no=emp.dept_no);
```

APELLIDO	SALARIO
NEGRO	3005
GIL	3000
REY	4100
FERNANDEZ	3000

Aclaración: Aunque hay sólo 3 departamentos con empleados (los nº 10, 20 y 30), se muestran 4 empleados porque en el departamento de Investigación (nº 20) el mayor sueldo (3000 €) lo tienen 2 empleados (Gil y Fernández).

Para más información sobre consultas correlacionadas con ejemplos explicados gráficamente se puede consultar:

<https://www.w3resource.com/sql/subqueries/corelated-subqueries-using-aliases.php>

SUBCONSULTAS Y JOINS

En algún ejemplo, una subconsulta también se puede reescribir utilizando JOINS. Casos generales, por ejemplo, **una consulta con IN puede reescribirse utilizando la cláusula DISTINCT:**

```
SELECT * FROM table1 WHERE col1 IN (SELECT col1 FROM table2);
```

Se convertiría en: `SELECT DISTINCT table1.* FROM table1, table2 WHERE table1.col1=table2.col1;`

O, las siguientes consultas con **NOT IN** ó **NOT EXISTS**, que obtendrían el mismo resultado:

```
SELECT * FROM table1 WHERE col1 NOT IN (SELECT col1 FROM table2);
```

```
SELECT * FROM table1 WHERE NOT EXISTS (SELECT col1 FROM table2 WHERE table1.col1=table2.col1);
```

También se podrían reescribir utilizando un **LEFT JOIN**, siendo a vez **más eficiente** esta última consulta:

```
SELECT table1.* FROM table1 LEFT JOIN table2 ON table1.id=table2.id WHERE table2.id IS NULL;
```

***HACER DEL EJERCICIO 38 AL 51**

***HACER EJERCICIOS FINAL DEL
TEMA (DEL 52 AL 11)**

8. Anexos

8.1. Ejemplos de combinación de tablas

Sean las siguientes tablas:

Tabla ASIGNATURAS

ASIGNATURA	Aula	CODLIBRO
GBD	C5	1002
Sistemas O.	C8	221
Sistemas 2	C7	
Sistemas 1	C7	221
Programación	C1	100

Tabla LIBROS

CODLIBRO	NOMBRE	EDITORIAL
100	Programación Java	Mc Graw Hill
221	WINDOWS 7	Ra-ma
301	Análisis	Ra-ma
333	Certificación CISCO	CISCO
1002	Introducción SQL	Info Hoy

Vemos las siguientes consultas sobre estas tablas:

*SELECT * FROM ASIGNATURAS, LIBROS;*

```
select * from asignaturas, libros;
```

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	100	Programación Java	Mc Graw Hill
GBD	C5	1002	221	WINDOWS 7	Ra-ma
GBD	C5	1002	301	Análisis	Ra-ma
GBD	C5	1002	333	Certificación CISCO	CISCO
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	100	Programación Java	Mc Graw Hill
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	301	Análisis	Ra-ma
Sistemas O.	C8	221	333	Certificación CISCO	CISCO
Sistemas O.	C8	221	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	100	Programación Java	Mc Graw Hill
Sistemas 2	C7	-	221	WINDOWS 7	Ra-ma
Sistemas 2	C7	-	301	Análisis	Ra-ma
Sistemas 2	C7	-	333	Certificación CISCO	CISCO
Sistemas 2	C7	-	1002	Introducción SQL	Info Hoy
Sistemas 1	C7	221	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	301	Análisis	Ra-ma
Sistemas 1	C7	221	333	Certificación CISCO	CISCO
Sistemas 1	C7	221	1002	Introducción SQL	Info Hoy
Programación	C1	100	100	Programación Java	Mc Graw Hill
Programación	C1	100	221	WINDOWS 7	Ra-ma
Programación	C1	100	301	Análisis	Ra-ma
Programación	C1	100	333	Certificación CISCO	CISCO
Programación	C1	100	1002	Introducción SQL	Info Hoy

25 filas devueltas en 0,00 segundos Exportación de CSV

8. Anexos

8.1. Ejemplos de combinación de tablas

*SELECT * FROM ASIGNATURAS CROSS JOIN LIBROS;*

```
select * from asignaturas cross join libros;
```

Resultados

Explicar

Describir

SQL Guardado

Historial

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	100	Programación Java	Mc Graw Hill
GBD	C5	1002	221	WINDOWS 7	Ra-ma
GBD	C5	1002	301	Análisis	Ra-ma
GBD	C5	1002	333	Certificación CISCO	CISCO
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	100	Programación Java	Mc Graw Hill
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	301	Análisis	Ra-ma
Sistemas O.	C8	221	333	Certificación CISCO	CISCO
Sistemas O.	C8	221	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	100	Programación Java	Mc Graw Hill
Sistemas 2	C7	-	221	WINDOWS 7	Ra-ma
Sistemas 2	C7	-	301	Análisis	Ra-ma
Sistemas 2	C7	-	333	Certificación CISCO	CISCO
Sistemas 2	C7	-	1002	Introducción SQL	Info Hoy
Sistemas 1	C7	221	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	301	Análisis	Ra-ma
Sistemas 1	C7	221	333	Certificación CISCO	CISCO
Sistemas 1	C7	221	1002	Introducción SQL	Info Hoy
Programación	C1	100	100	Programación Java	Mc Graw Hill
Programación	C1	100	221	WINDOWS 7	Ra-ma
Programación	C1	100	301	Análisis	Ra-ma
Programación	C1	100	333	Certificación CISCO	CISCO
Programación	C1	100	1002	Introducción SQL	Info Hoy

25 filas devueltas en 0.00 segundos

Exportación de CSV

*SELECT *
FROM ASIGNATURAS A, LIBROS L
WHERE A.CODLIBRO=L.CODLIBRO;*

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy

4 filas devueltas en 0,02 segundos [Exportación de CSV](#)

8. Anexos

8.1. Ejemplos de combinación de tablas

*SELECT * FROM ASIGNATURAS
NATURAL JOIN LIBROS;*

CODLIBRO	ASIGNATURA	AULA	NOMBRE	EDITORIAL
100	Programación	C1	Programación Java	Mc Graw Hill
221	Sistemas 1	C7	WINDOWS 7	Ra-ma
221	Sistemas O.	C8	WINDOWS 7	Ra-ma
1002	GBD	C5	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

*SELECT * FROM ASIGNATURAS
INNER JOIN LIBROS USING (CODLIBRO);*

CODLIBRO	ASIGNATURA	AULA	NOMBRE	EDITORIAL
100	Programación	C1	Programación Java	Mc Graw Hill
221	Sistemas 1	C7	WINDOWS 7	Ra-ma
221	Sistemas O.	C8	WINDOWS 7	Ra-ma
1002	GBD	C5	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

*SELECT * FROM ASIGNATURAS JOIN LIBROS ON
ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;*

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy

4 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

*SELECT * FROM ASIGNATURAS
LEFT OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
Salen todas las asignaturas*

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	-	-	-

5 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

8. Anexos

8.1. Ejemplos de combinación de tablas

```
SELECT * FROM ASIGNATURAS
RIGHT OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
```

Salen todos los libros

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Programación	C1	100	100	Programación Java	Mc Graw Hill
-	-	-	333	Certificación CISCO	CISCO
-	-	-	301	Análisis	Ra-ma

```
SELECT * FROM ASIGNATURAS
FULL OUTER JOIN LIBROS
ON ASIGNATURAS.CODLIBRO=LIBROS.CODLIBRO;
```

Salen todos los libros y las asignaturas.

ASIGNATURA	AULA	CODLIBRO	CODLIBRO	NOMBRE	EDITORIAL
Programación	C1	100	100	Programación Java	Mc Graw Hill
Sistemas 1	C7	221	221	WINDOWS 7	Ra-ma
Sistemas O.	C8	221	221	WINDOWS 7	Ra-ma
GBD	C5	1002	1002	Introducción SQL	Info Hoy
Sistemas 2	C7	-	-	-	-
-	-	-	333	Certificación CISCO	CISCO
-	-	-	301	Análisis	Ra-ma

7 filas devueltas en 0,00 segundos

[Exportación de CSV](#)

8.2. Tablas de los ejercicios

EMPLE

EMP_NO	APELLIDO	OFICIO	DIR	FECHA_ALT	SALARIO	COMISION	DEPT_NO
7369	SANCHEZ	EMPL...	7902	1990-12-17...	1040	NULL	20
7499	ARROYO	VEN...	7698	1990-02-20...	1500	390	30
7521	SALA	VEN...	7698	1991-02-22...	1625	650	30
7566	JIMENEZ	DIRE...	7839	1991-04-02...	2900	NULL	20
7654	MARTIN	VEN...	7698	1991-09-29...	1600	1020	30
7698	NEGRO	DIRE...	7839	1991-05-01...	3005	NULL	30
7782	CEREZO	DIRE...	7839	1991-06-09...	2885	NULL	10
7788	GIL	ANA...	7566	1991-11-09...	3000	NULL	20
7839	REY	PRES...	NULL	1991-11-17...	4100	NULL	10
7844	TOVAR	VEN...	7698	1991-09-08...	1350	0	30
7876	ALONSO	EMPL...	7788	1991-09-23...	1430	NULL	20
7900	JIMENO	EMPL...	7698	1991-12-03...	1335	NULL	30
7902	FERNA...	ANA...	7566	1991-12-03...	3000	NULL	20
7934	MUÑOZ	EMPL...	7782	1992-01-23...	1690	NULL	10

DEPART

DEPT_NO	DNOMBRE	LOC
10	CONTABILIDAD	SEVILLA
20	INVESTIGACION	MADRID
30	VENTAS	BARCELONA
40	PRODUCCION	BILBAO

8. Anexos

8.2. Tablas de los ejercicios

ALUM1213

DNI	NOMBRE	APELLIDOS	FECHA_NAC	DIRECCION	POBLACION	PROVINCIA	CURSO	NIVEL	CLASE	FALTAS1	FALTAS2	FALTAS3
16432432J	Ana	Martínez Escobés	1994-04-03 00:00:00	C/ Madrid, 12	Lardero	La Rioja	1	ASIR	V	3	8	1
16562562W	Eduardo	Bravo López	1989-02-02 00:00:00	C/ Clavijo, 40	Logroño	La Rioja	2	DAM	D	5	2	3
16654654A	Luis Javier	Martínez Lajo	1993-03-03 00:00:00	C/ Bretón, 14	Logroño	La Rioja	2	DAM	D	1	0	1
16789789A	Eduardo	Castor Flores	1989-10-08 00:00:00	C/ Los almendros, 4	Villamediana	La Rioja	1	DAM	D	0	0	0
34448974N	CONCHA	ESTRUCH VIDAL	1985-11-10 00:00:00	C/ MEDICO FELIX, 12	BERROCALEJO	CACERES	2	DAM	D	0	0	0
34448999N	DORA	MARTIN BREÑA	1994-11-10 00:00:00	C/ MEDICO FELIX, 10	CACERES	CACERES	1	ASIR	D	0	0	0

ASIGNATURAS

COD	NOMBRE
1	Programación
2	Sist. Informáticos
3	Bases de datos
4	FOL
5	Inglés técnico
6	Lenguajes de marcas
7	Seguridad
8	Entornos

ALUMNOS

DNI	APENOM	DIREC	POBLA	TELEF
16344345	Alcalde García, Elena	C/Las Matas, 24	Madrid	917766545
16448242	Cerrato Vela, Luis	C/Mina 28 - 3A	Madrid	916566545
16882942	Díaz Fernández, María	C/Luis Vives 25	Móstoles	915577545

NOTAS

DNI	COD	NOTA
16344345	1	6
16344345	2	5
16344345	3	6
16344345	4	5
16344345	5	6
16344345	6	8
16344345	7	4
16448242	4	6
16448242	5	8
16448242	6	4
16448242	7	5
16882942	4	8
16882942	5	7
16882942	6	8
16882942	7	9

NOTAS_ALUMNOS

NOMBRE_ALUMNO	NOTA1	NOTA2	NOTA3
Alcalde García, M. Luisa	5	5	5
Benito Martín, Luis	7	6	8
Casas Martínez, Manuel	7	5	5
Casas Sánchez, Ana	6	9	8
Díaz Sánchez, María	NULL	NULL	7
Martín Salas, Fernando	6	5	7
Sánchez Fuentes, Susana	4	5	4
Jover Sánchez, Mario	7	7	5
Lafuente Sánchez, Sara	6	6	6
Sánchez Lahera, Fernando	3	3	6

8. Anexos

8.2. Tablas de los ejercicios

LIBRERÍA

TEMA	ESTANTE	EJEMPLARES
INFORMATICA	A	15
ECONOMIA	A	10
DEPORTES	B	8
FILOSOFIA	C	7
DIBUJO	C	10
MEDICINA	C	16
BIOLOGIA	A	11
GEOLOGIA	D	7
SOCIEDAD	D	9
LABORES	B	20
JARDINERIA	E	6

LECTOR

NUMLECTOR	NOMLECTOR	TIPOLECTOR
1	Posadas Gil, Inés	3
2	Sánchez Pons, José	3
3	Gómez Sáez, Miguel	3
10	Santana Páez, Eva	4
11	Betancor Díaz, Yolanda	4
20	Blasco Ulla, Juan Luis	1
22	Marín Gómez, Ana	1
30	Plaza Ortiz, Juan	8

LIBRO

CODLIBRO	TITULO	NUMPAG	EDITORIAL	AUTOR
12	Don Quijote I	517	Anaya	Cervantes
13	Don Quijote II	611	Anaya	Cervantes
20	El principito	120	Andina	Saint_Exupery
35	Fortunata y Jacinta	625	PlazaJanes	Pérez-Galdós
40	Caperucuta en Manhattan	310	Salamandra	Martín GAite
41	La piel del tambor	280	Circulo	Pérez-Reverte
42	Afrodita	421	Espasa	Isabel Allende
50	Viajar	675	BBVA	NULL
51	España en sus caminos	534	BBVA	NULL
52	Aeroguía del litoral	340	Planeta	NULL
65	Don Quijote I	578	Planeta	Cervantes
66	Don Quijote II	600	Planeta	Cervantes

PRSTAMO

CODLIBRO	NUMLECTOR	FPRESTA	DIAS
12	20	2012-04-12 00:00:00	54
13	11	2012-05-10 00:00:00	26
20	11	2012-06-01 00:00:00	4
35	1	2012-04-15 00:00:00	51
40	3	2011-11-01 00:00:00	217
42	30	2012-05-05 00:00:00	31
50	20	2012-03-05 00:00:00	92
51	20	2012-05-06 00:00:00	30
52	22	2012-05-06 00:00:00	30